

Aula Prática

Nomes:

Beatriz de Lara Gelki Arduim
Eloisa Formehl de Alencar
Guilherme Ferreira da Silva
Heitor Corrêa Medardo
Matheus Yhudi Busemeier Fermiano
Pedro Henrique Alves Habizenreuter

Turma: V02

Disciplina: Teste de sistemas

1. Testes Não Funcionais (Performance, Carga, Estresse)

A. Introdução (Porquê e Como)

Testes Não Funcionais (Performance, Carga, Estresse)

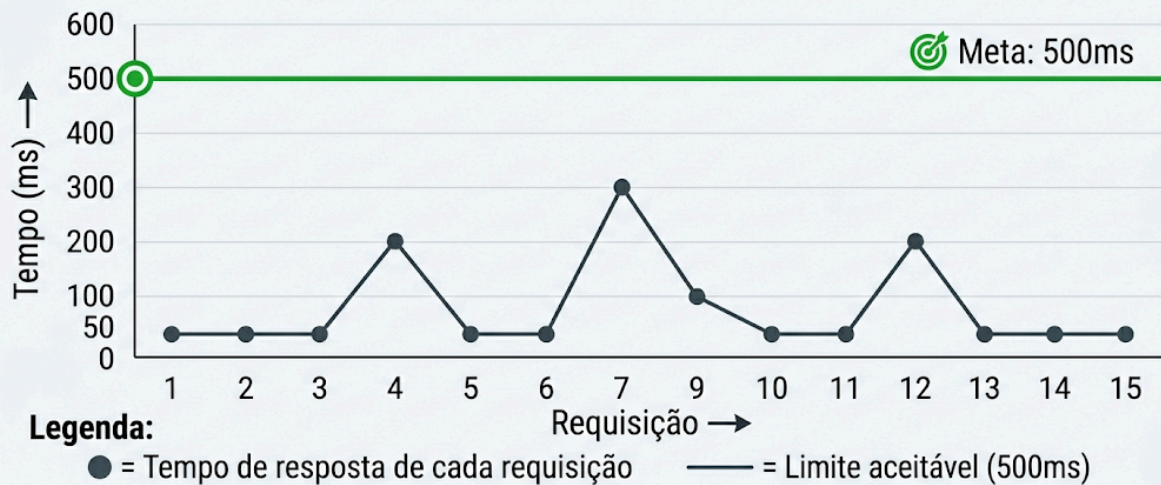
Porquê: Realizamos testes não funcionais porque não basta um software ter todas as suas funcionalidades corretas se ele não consegue sobreviver ao mundo real sob pressão. Enquanto os testes funcionais garantem a lógica do sistema (por exemplo, se o botão de finalizar compra funciona), os testes não funcionais focam na experiência do usuário e na estabilidade da aplicação. O objetivo principal é descobrir os limites do sistema para evitar lentidões extrema, falhas ou quedas totais durante picos de acesso (como em uma Black Friday), protegendo a reputação da empresa e evitando a perda de vendas por frustração do cliente.

Como: Para realizar esses testes, utilizamos ferramentas especializadas, como JMeter, Gatling ou K6, que geram milhares de "usuários virtuais" para simular acessos simultâneos em diferentes intensidades. Durante essa simulação, monitoramos os sinais vitais do sistema, como o *Throughput* (quantas requisições são processadas por segundo) e a *Latência* (o tempo que a resposta demora para chegar). Aumentamos o volume de acessos intencionalmente, passando da performance normal para a carga máxima esperada e, por fim, para o estresse total, até o sistema quebrar. O trabalho real acontece ao analisar esses dados para encontrar o "Gargalo" (Bottleneck) — identificando se o que limitou ou derrubou o sistema foi o banco de dados, o servidor, a rede ou o processamento.

B. Visualização Gráfica do Tópico

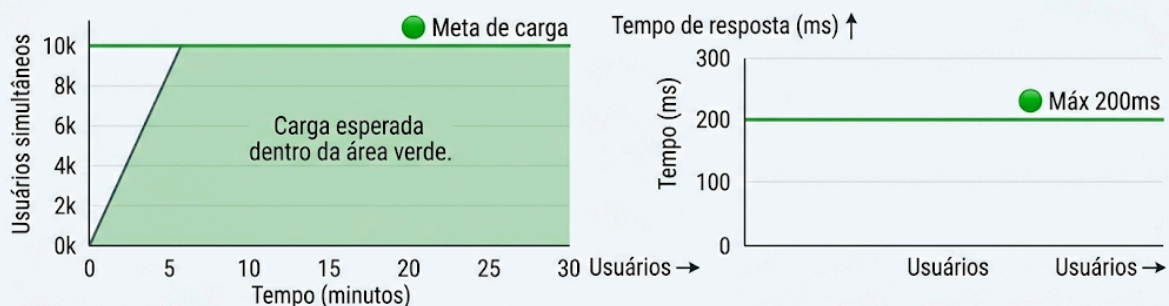
Abaixo estão os gráficos que ilustram como o sistema se comporta à medida que aumentamos a quantidade de usuários, destacando as três fases do teste e o momento em que o gargalo é atingido.

TESTE DE PERFORMANCE - Tempo de Resposta por Requisição

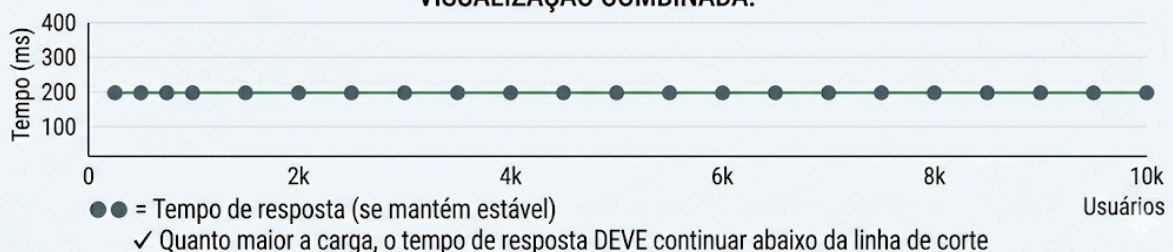


✓ **Resultado:** Nenhuma ● ultrapassou a linha, todas abaixo de 500ms → **Aprovado**

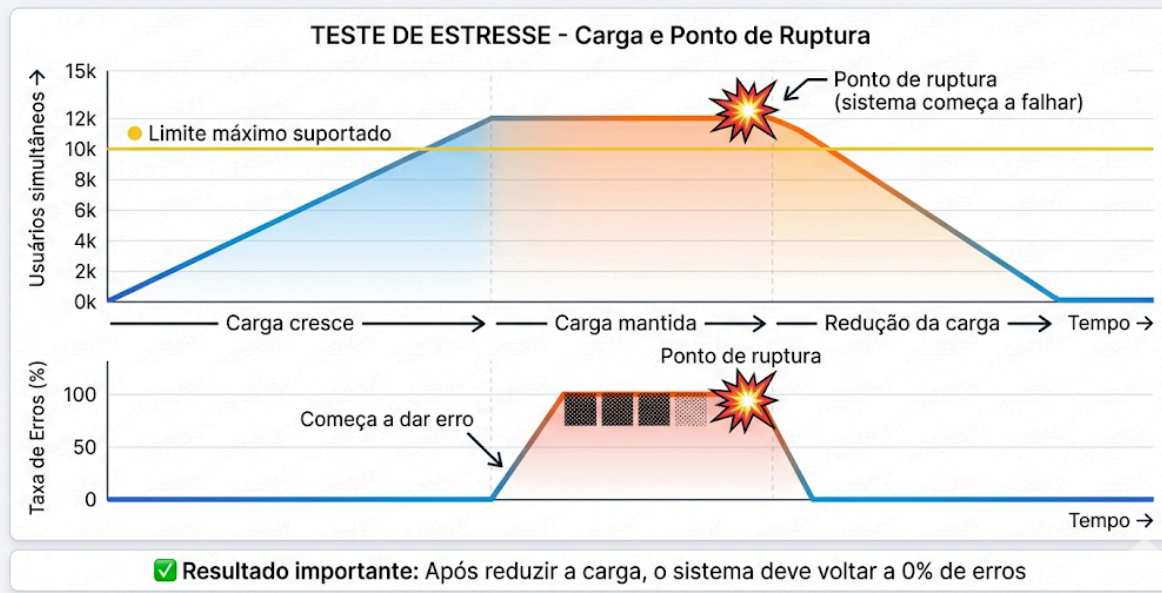
TESTE DE CARGA - Usuários x Tempo de Resposta



VISUALIZAÇÃO COMBINADA:



TESTE DE ESTRESSE - Até a Quebra e Recuperação



C. 5 perguntas desafiadoras sobre o tópico;

1. "Por que não basta testar o sistema com apenas uma pessoa usando? Se funciona para um, não deveria funcionar para mil?"

Resposta: Funciona para um, mas para mil podem surgir problemas de concorrência (fila de requisições), esgotamento de conexões com o banco de dados, ou consumo excessivo de memória RAM e CPU que não aparecem com um único usuário. É como uma porta estreita: uma pessoa passa rápido, 1000 formam uma aglomeração.

2. Qual a diferença entre teste de performance, carga e estresse?

Resposta: Performance: Mede a velocidade do sistema.

Carga: Testa com o volume esperado de usuários.

Estresse: Testa até quebrar, para ver o limite e a recuperação.

3. Um sistema pode "morrer" de repente no teste de estresse sem dar nenhum aviso antes?

Sim. É comum o sistema funcionar bem até um certo limite e, de repente, travar completamente. Isso acontece quando algum recurso (memória, conexões de banco) se esgota de forma abrupta, como um copo que transborda.

4. Dá para fazer teste de carga sem ferramenta especializada, só com pessoas reais clicando ao mesmo tempo?

Não. Seria impossível sincronizar centenas ou milhares de pessoas no mesmo segundo, e o custo seria absurdo. As ferramentas simulam esse exército de usuários virtuais com precisão de milissegundos.

5. Teste de performance é só sobre velocidade?

Não. Envolve também estabilidade (não oscilar), escalabilidade (crescer com mais recursos) e uso eficiente de CPU/memória. Um sistema rápido que consome 100% do servidor não é performático.

D. 3 exemplos de teste contendo, Cenário, Como

Exemplo 1: Teste de Carga

- **Cenário:** Uma loja online espera 1000 clientes simultâneos durante uma promoção normal de fim de mês.
- **Como Testar:** Usar uma ferramenta (ex: JMeter) simulando 1000 usuários virtuais navegando, pesquisando produtos e finalizando compras ao mesmo tempo.
- **Objetivo:** Verificar se o site mantém tempo de resposta aceitável e não trava com o movimento esperado.

Exemplo 2: Teste de Estresse

- **Cenário:** A mesma loja online quer saber qual o limite máximo antes de sair do ar.
- **Como Testar:** Aumentar gradualmente os usuários virtuais (2000, 3000, 5000...) até o sistema começar a dar erro ou cair.
- **Objetivo:** Descobrir o ponto de ruptura e, principalmente, ver se o sistema volta sozinho depois do pico.

Exemplo 3: Teste de Performance

- **Cenário:** Um aplicativo de banco precisa exibir o saldo do cliente.
- **Como Testar:** Medir o tempo entre o clique em "Ver Saldo" e a tela carregar completamente, com apenas 1 usuário usando.

- **Objetivo:** Garantir que a resposta seja rápida (ex: menos de 2 segundos) e estável, sem oscilações.

2. Testes de Usabilidade e Acessibilidade Digital (UX)

Descrição:

Focado na interação humano-computador.

A exposição aqui deve cobrir métodos para avaliar se a interface é intuitiva, eficiente e agradável para o usuário comum (Usabilidade), além de garantir que o software possa ser utilizado por pessoas com deficiências visuais, motoras ou cognitivas, cumprindo requisitos legais e éticos de mercado (Acessibilidade).

Termos ideais de pesquisa:

As 10 Heurísticas (explicadas de forma simples):

1. **Visibilidade do status do sistema**
O sistema deve sempre informar o que está acontecendo (ex: loading, sucesso)
2. **Correspondência com o mundo real**
Linguagem simples, sem “tecnês”
3. **Controle e liberdade do usuário**
Permitir desfazer/refazer ações
4. **Consistência e padrões**
Não reinventar o que já é padrão (botões, menus)
5. **Prevenção de erros**
Evitar que o erro aconteça (ex: validação de formulário)
6. **Reconhecimento em vez de memorização**
Interfaces intuitivas (menus visíveis, sugestões)
7. **Flexibilidade e eficiência de uso**
Atalhos para usuários avançados
8. **Design minimalista**
Sem excesso de informação
9. **Ajudar a reconhecer e corrigir erros**
Mensagens claras de erro
10. **Ajuda e documentação**
Suporte acessível quando necessário.

O que é WCAG:

-são diretrizes internacionais de acessibilidade digital criadas pelo W3C para tornar sites, aplicativos e conteúdos web utilizáveis por pessoas com deficiências.

WCAG organiza regras em 4 princípios (POUR):

1. Perceptível

- Conteúdo deve ser visível/audível
- Ex: texto alternativo em imagens

2. Operável

- Interface deve funcionar via teclado
- Ex: navegação sem mouse

3. Compreensível

- Conteúdo claro e previsível

4. Robusto

- Compatível com tecnologias assist

Tipos de Teste de Usabilidade:

Teste com usuários reais:

- Usuários executam tarefas reais
- Observa-se comportamento, erros e dificuldades
- Mais realista, porém mais caro e demorado

Teste heurístico:

- Especialistas avaliam com base nas heurísticas
- Mais rápido e barato
- Pode não refletir totalmente o comportamento real

Ferramentas para testes de acessibilidade

Leitores de tela (essenciais):

- NVDA
- VoiceOver

Para que servem?

Simulam a experiência de usuários com deficiência visual:

- Leem conteúdo da tela
- Testam navegação por teclado
- Revelam problemas invisíveis para quem enxerga

Foco Essencial:

Diretrizes de Acessibilidade Web (WCAG).

Por quê?

Enquanto a usabilidade pode parecer subjetiva para iniciantes, o padrão WCAG (Web Content Accessibility Guidelines) traz regras testáveis, binárias (passou/falhou) e técnicas, facilitando a elaboração de roteiros de testes claros para a acessibilidade.

Visualização gráfica:

🧩 Testes de Usabilidade e Acessibilidade (UX)

🎯 Usabilidade

Interface fácil, eficiente e intuitiva

- Status do sistema
- Linguagem simples
- Controle do usuário
- Consistência
- Prevenção de erros
- Interface intuitiva
- Eficiência (atalhos)
- Design simples
- Tratamento de erros
- Ajuda e documentação

🔧 Testes de Usabilidade

- **Usuários reais:** mais realista, porém caro
- **Heurístico:** rápido e barato

👤 Acessibilidade (WCAG)

Diretrizes internacionais para inclusão digital

Perceptível	Operável
Compreensível	Robusto

Ex: alt text, teclado, clareza, compatibilidade

🛠️ Ferramentas

- NVDA
- VoiceOver

- ✓ Leem a tela
- ✓ Navegação por teclado
- ✓ Detectam problemas invisíveis

3. Fundamentos de Segurança da Informação nos Testes

Descrição:

Introdução às práticas de garantia de segurança (Quality Assurance voltado para Security). Vocês devem mapear e testar vulnerabilidades comuns na camada de aplicação, validando se o sistema protege os dados do usuário, se as autenticações são robustas e se as entradas de dados estão blindadas contra injeções de código malicioso.

Termos ideais de pesquisa:

"OWASP Top 10 vulnerabilidades web atualizado"

"SQL Injection e Cross-Site Scripting (XSS) como testar"

"Diferença entre SAST, DAST e Pentest manual"

"Testes de validação de input de dados"

Foco Essencial: OWASP Top 10.

Por quê?

A OWASP (Open Worldwide Application Security Project) é a principal referência do mercado. Focar no top 10 direciona a atenção para os riscos cibernéticos mais críticos e reais, transformando a teoria abstrata de segurança em um checklist prático de testes.

1. Introdução: O Porquê e o Como

O Porquê: Antigamente, o QA só testava se o botão funcionava. Hoje, isso não basta. Segurança é prioridade porque um vazamento de dados pode destruir a reputação de uma empresa e gerar multas pesadas (como a LGPD). Testar segurança desde o início evita que o erro chegue ao usuário final, onde o prejuízo é muito maior.

O Como: Fazemos isso saindo do "caminho feliz" do teste. Em vez de apenas preencher um formulário, tentamos "quebrá-lo" inserindo códigos onde deveriam ir apenas textos, testando se o sistema aceita senhas fracas ou se permite que um usuário veja dados de outro apenas mudando um número na URL.

2. Visualização Gráfica (Sugestão para o slide)

Imagine um "Escudo de Camadas":

1. **Camada Externa (Input):** Validação de dados (filtros).
2. **Camada Média (Aplicação):** Autenticação robusta (login e senhas).
3. **Camada Interna (Banco de Dados):** Criptografia e proteção contra injeções. *O centro do escudo é o Dado do Usuário.*

3. 5 Perguntas Desafiadoras

1. Se o sistema bloqueia caracteres especiais no Front-end, ele está 100% seguro contra SQL Injection? (Resposta: Não, a validação deve ser no Back-end).
2. Qual a diferença entre "esqueci minha senha" seguro e um inseguro?
3. Como um atacante pode roubar sua sessão de login usando apenas um campo de comentário? (XSS).
4. Por que usar bibliotecas de terceiros desatualizadas é um risco de segurança?
5. O que é mais perigoso: um erro que mostra o código do servidor ou um que não mostra nada?

4. Exemplos de Teste Prático

Exemplo 01: Injeção de SQL

- **Cenário:** Tela de login.
- **Como Testar:** Digitar ' OR 1=1 -- no campo de usuário.
- **Objetivo:** Garantir que o sistema não ignore a senha e dê acesso indevido.

Exemplo 02: Cross-Site Scripting (XSS)

- **Cenário:** Cadastro de nome de perfil.
- **Como Testar:** Salvar o nome como `<script>alert('oi')</script>`.
- **Objetivo:** Validar se o sistema "limpa" o texto ou se executa o código (o que seria uma falha).

Exemplo 03: Controle de Acesso

- **Cenário:** Visualização de pedido (ex: site.com/pedido/10).
- **Como Testar:** Alterar o número final para **11** sendo o Usuário 10.
- **Objetivo:** Impedir que um cliente veja o pedido de outro.

A pergunta que escolhemos foi a 10:

10. Qual é a diferença conceitual e prática entre as abordagens de avaliação de segurança SAST, DAST e o Pentest (teste de intrusão) manual?

SAST: É o "Raio-X". Lê o **código-fonte** (com o sistema desligado) para encontrar erros de programação.

- **DAST:** É o teste de "Caixa Preta". Testa o **sistema em execução**, jogando dados ruins nos campos para ver se ele quebra.
- **Pentest (Teste de Intrusão):** É o teste **humano**. Um hacker ético usa a criatividade para explorar falhas na lógica do sistema que as ferramentas automáticas (SAST/DAST) não conseguem identificar.